

[Home](#) / [Community](#) / [Architectures And Processors Blog](#) /

May 29, 2016 5月29日

Exploring How Cache Coherency Accelerates Heterogeneous Compute

探索缓存一致性如何加速异构计算

This blog focuses on some of the hardware innovations and changes that are relevant to shared virtual memory and cache coherency, which are components of the HSA hardware specification.

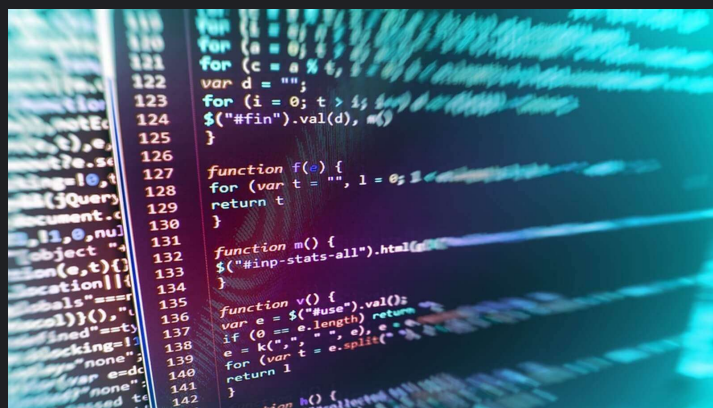
这篇博客关注的是一些与共享虚拟内存和缓存一致性相关的硬件创新和变化，它们是HSA硬件规范的组成部分。

By [Neil Parris](#) 尼尔·帕里斯

Share
分享



Reading time 11 minutes 阅读时间11分钟



Cache Coherency and Shared Virtual Memory

缓存一致性和共享虚拟内存

The [Heterogeneous System Architecture \(HSA\) Foundation](#) is a not-for profit consortium for SoC IP vendors, OEMs, academia, SoC vendors, OSVs and ISVs whose goal is to make it easier for software developers to take advantage of all the advanced processing hardware on a modern SoC. The CPU and GPU on a

applications can improve the end user experience. Done right, efficiency can be gained in power, performance, programmability and portability.

异构系统架构（HSA）基金会是由SoC IP供应商、oem厂商、学术界、SoC供应商、osv和isv组成的非营利性联盟，其目标是使软件开发人员更容易在现代SoC上利用所有先进的处理硬件。典型应用处理器上的CPU和GPU占据了芯片面积的很大一部分，高效地将这些资源应用于多个应用可以提升最终用户体验。如果处理得当，可以在功能、性能、可编程性和可移植性方面提高效率。

This blog focuses on some of the hardware innovations and changes that are relevant to shared virtual memory and cache coherency, which are components of the HSA hardware specification.

这篇博客关注的是一些与共享虚拟内存和缓存一致性相关的硬件创新和变化，它们是HSA硬件规范的组成部分。

What is Shared Virtual Memory?

什么是共享虚拟内存？

Traditional memory systems defined separate memory for CPU and GPU. In the case of PCs, the GPU may have completely separate discrete memory chips on a different board. In these systems, any application that wants to share data between CPU and GPU will need to copy it from CPU memory to graphics memory at a significant cost of latency and power.

传统的内存系统为CPU和GPU定义了独立的内存。在pc的情况下，GPU可能在不同的板上有完全独立的内存芯片。在这些系统中，任何想要在CPU和GPU之间共享数据的应用程序都需要将数据从CPU内存复制到图形内存，这将以显著的延迟和功耗成本为代价。

Mobile systems have had a unified memory system for many years where all processors can access the same physical memory. However, even though this is physically possible, the software APIs and memory management hardware and software may not allow this. Graphics buffers may still be defined separately from other memory regions and data sharing may still require an expensive copy of data between buffers.

移动系统多年来一直有一个统一的内存系统，所有处理器都可以访问相同的物理内存。然而，即使这在物理上是可能的，但软件api和内存管理硬件和软件可能不允许这样做。图形缓冲区仍然可能与其他内存区域分开定义，数据共享仍然可能需要在缓冲区之间复制昂贵的数据。

Shared virtual memory (SVM) allows processors to see the same view of memory; specifically, the same virtual address on the CPU and GPU will point to the same physical memory location. With this architecture, an application only needs to pass a

说，CPU和GPU上相同的虚拟地址将指向相同的物理内存位置。在这种体系结构中，应用程序只需要在共享数据的处理器之间传递一个指针。

There are multiple ways to implement SVM; it doesn't mean you have to share the exact same page table. The only requirement is that if a buffer is to be shared between processors then it must appear in the page tables for both memory management units (MMUs). With SVM in place, sharing data becomes as simple as passing a pointer between processors.

SVM有多种实现方法；这并不意味着您必须共享完全相同的页表。唯一的要求是，如果一个缓冲区在处理器之间共享，那么它必须出现在两个内存管理单元（mmu）的页表中。有了SVM，共享数据就像在处理器之间传递指针一样简单。

So What is Cache Coherency? 那么什么是缓存一致性呢？

Let's go back to basics and ask what does coherency mean?

Coherency is about ensuring all processors, or bus masters in the system see the same data. For example if I have a processor which is creating a data structure in its local cache then passing it to a GPU, both the processor and GPU must see the same data. If the GPU reads from external DDR, the GPU will read old, stale data.

让我们回到基础，问问什么是一致性？一致性是指确保系统中所有的处理器或总线主节点都能看到相同的数据。例如，如果我有一个处理器，它在本地缓存中创建一个数据结构，然后将其传递给GPU，处理器和GPU必须看到相同的数据。如果GPU从外部DDR读取数据，那么GPU将读取陈旧的数据。

There are three mechanisms to maintain coherency:

有三种机制来保持一致性：

- **Disable caching** is the simplest mechanism but may cost significant processor performance. To get the highest performance, processors are pipe-lined to run at high frequency and access caches which offer a very low latency. Caching of data that is accessed multiple times increases performance significantly and reduces DRAM accesses and power. Marking data as "non-cached" could impact performance and power and in reality is not used.

禁用缓存是最简单的机制，但可能会显著降低处理器性能。为了获得最高的性能，处理器以流水线方式以高频率运行，并访问延迟非常低的缓存。对多次访问的数据进行缓存，可以显著提高性能，减少DRAM访问次数和功耗。将数据标记为非缓存会影响性能和功耗，实际上是不会被使用的。

- **Software managed coherency** is the traditional solution to the data sharing problem. Here the software, usually device drivers, must clean dirty data from caches and invalidate old data to enable sharing with other processors or masters in the system. This takes processor cycles, bus bandwidth, and

(通常是设备驱动程序) 必须清理高速缓存中的脏数据, 使旧数据失效, 以便与系统中的其他处理器或主处理器共享数据。这需要处理器周期、总线带宽和电源。

- **Hardware managed coherency** offers an alternative to simplify software. With this solution any cached data marked 'shared' **will** always be up to date, automatically. All processors and bus masters in that sharing domain see the exact same value.

硬件管理的一致性为简化软件提供了另一种选择。使用此解决方案, 任何标记为共享的缓存数据将始终自动更新。该共享域中的所有处理器和总线主控器都会看到完全相同的值。

Challenges with Software Coherency

软件一致性的挑战

A cache stores external memory contents close to the processor to reduce the latency and power of accesses. On-chip memory accesses are significantly lower power than external DRAM accesses.

cache将外部存储器的内容存储在靠近处理器的地方, 以减少访问的延迟和功耗。片上存储器访问功耗明显低于外部DRAM访问功耗。

Software managed coherency manages cache contents with two key mechanisms:

软件管理的一致性通过两个关键机制管理缓存内容:

- Cache Cleaning:
 - If any data stored in a cache is modified, it is marked as 'dirty' and must be written back to DRAM at some point in the future. The process of cleaning **will** force dirty data to be written to external memory. There are two ways to do this: 1) clean the whole cache which would impact **all** applications, or 2) clean specific addresses one by one. Both are very expensive in CPU cycles.
 - With modern multi-core systems this cache cleaning must happen on **all** cores.
- Cache Invalidation:
 - If a processor has a **local** copy of data, but an external agent updates main memory then the cache contents are out of date, or 'stale'. Before reading this data the processor must remove the stale data from caches, this is known as 'invalidation' (a cache line is marked invalid).
 - An example is a region of memory used as a shared buffer for network traffic which may be updated by a network interface DMA hardware; a processor wishing to access this data must invalidate any old stale copy before reading the new data.

Coherency

"We would like to connect more devices with hardware coherency to simplify software and accelerate product schedules"

"50% of debug time is spent on SW coherency issues as these are difficult to find and pinpoint"

Quotes from a system architect at an application processor vendor.

Software coherency is hard to debug; the cache cleaning and invalidation must be done at the right time. If done too often it wastes power and CPU effort. If done too infrequently it will result in stale data which may cause unpredictable application behaviour, if not a crash. Debugging this is extremely difficult as it will present occasional data corruption.

Looking specifically at CPU and GPU sharing, this software cache maintenance will be difficult to optimize and applications on these systems will try and avoid sharing data due to cost and complexity. One middleware vendor using GPU compute with software coherency noted that the developers spent around 30% of their time architecting, implementing and debugging the data sharing including breaking down image data into sub-frames and careful timing of the mapping and unmapping functions.

When sharing is used with software coherency, the size of the task running on the GPU must be large enough to make it worthwhile, taking into account the cost of software coherency.

Hardware Coherency Requires an Advanced Bus Protocol

Extending hardware coherency to the system requires a coherent bus protocol, and in 2011 ARM released the [AMBA 4 ACE](#) specification which introduces the "AXI Coherency Extensions" on top of the popular AXI protocol. The full ACE interface allows hardware coherency between processor clusters and allows an SMP operating system to extend to more cores.

With the example of two clusters, any shared access to memory can 'snoop' into the other cluster's caches to see if the data is already on chip; if not, it is fetched from external memory (DDR). In mobile, this has enabled the [big.LITTLE](#) processing model which improves performance and power efficiency by utilizing the right core to suit the size of the task.

The AMBA 4 ACE-Lite interface is designed for IO (or one-way) coherent system masters like DMA engines, network interfaces and accelerators. These devices may not have any caches of their own, but they can read shared data from the ACE processors. Alternatively, they may have caches but these would still need to be cleaned and invalidated by software.

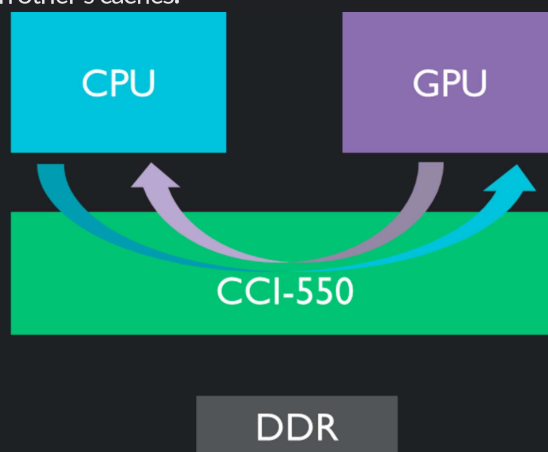
and enables applications that would not be possible with software coherency such as big.LITTLE processing.

Adding Hardware Coherency to the GPU

While processor clusters have implemented cache coherency protocols for many years, this is a new area for GPUs. As applications look to share more data between CPU and GPU, hardware cache coherency ensures this can be done at a low cost in power and latency, which in turn makes it easier, more power efficient and higher performance than any software managed mechanism. Most importantly it makes it easy for the software developer to share data.

There are two ways a GPU could be connected with hardware coherency:

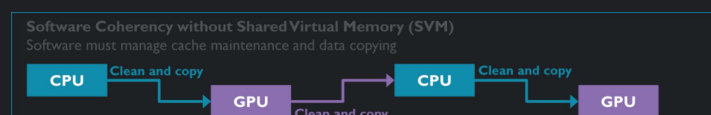
- IO coherency (also known as one-way coherency) using ACE-Lite where the GPU can read from CPU caches. Examples include the [ARM Mali-T600, 700 and 800 series GPUs](#).
- Full coherency using full ACE, where CPU and GPU can see each other's caches.



Full coherency between CPU and GPU, enables best use of available compute resource

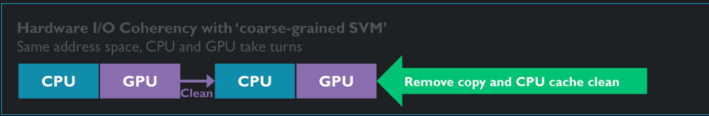
The Powerful Combination of SVM and Hardware Coherency

The following diagrams summarize what we've learned so far and also describe the coarse and fine grain shared virtual memory. These charts approximate elapsed time on the horizontal axis, and address space on the vertical axis.

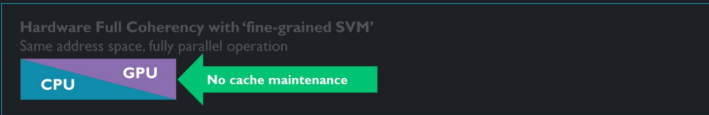




copied between processors to 'share' the data. In addition to cache cleaning, the target cache would also need to invalidate any old data before reading new data from DRAM. This is time consuming and power hungry and limits the applications that can take advantage of heterogeneous processing.



With shared virtual memory the CPU and GPU can now share physical memory and operate on the same virtual address, which eliminates the copy. If we have an IO coherent GPU, in other words one-way coherent where GPU can read CPU caches, then we remove the need to clean data from CPU caches. However, because this is one-way, the CPU cannot see the GPU caches. This means the GPU caches must be cleaned with cache maintenance operations after processing completes. This 'coarse-grain' SVM means the processors must take turns accessing the shared buffer.



Finally, if we enable a fully coherent memory system then both CPU and GPU can see exactly the same data at all times, and we can use 'fine-grained' SVM. This means both processes can access the same buffer at the same time instead of taking turns. Handshaking between processors uses cross-device atomics. By removing all of the cache maintenance overheads we can get the best overall performance.

Connecting Hardware with Software: Compute APIs

At this point it's useful to map these hardware technologies to the software APIs. Compute APIs like OpenCL 2.0 can take full advantage of SVM and hardware coherency, and can run on HSA platforms. Not all OpenCL 2.0 implementations are the same; there are a number of optional features that can be enabled if the hardware supports it. These features can also be mapped to the HSA profiles: base profile and full profile, as shown in the table below.

OpenCL Feature	Shared Virtual Memory	Fully Coherent Memory	HSA Profile
Fine Grained Buffer	Required, buffer level	Required, fully coherent	Base Profile
Fine Grained System	Required, full memory	Required, fully coherent	Full Profile



Grain buffer level coherency)

HSA always requires hardware coherency, and with the base profile the scope of shared virtual memory can be limited to the shared buffers. This means only the shared buffers would appear in both CPU and GPU page tables, not the full system memory. This may be easier and lower cost to implement in hardware.

Full coherency is required for fine grain, and this enables both CPU and GPU to work on different addresses with the same data buffer at the same time.

Full coherency also allows the use of atomic operations, which allows processors to work on the same address within the same buffer. Atomic operations allow synchronization between threads, much like in a multi-core CPU. Atomics are optional for OpenCL but required for HSA.

For coarse grain, if hardware coherency is not present then it would need to use software managed coherency including cache maintenance operations, or optionally IO coherency for the GPU.

Hardware Requirements for Cache Coherency and Shared Virtual Memory

The hardware required to implement these technologies already exists today in the form of fully coherent processors and cache coherent interconnects. The interconnect is responsible for connecting processors, peripherals and memory together on the SoC. The AMD Kavari APU already has a fully coherent memory between the CPU and GPU. ARM offers IP such as the [CoreLink CCI-550 Cache Coherent Interconnect](#), [Cortex-A72](#) processor and the [Mali-G71 GPU](#), which together support the full coherency and shared virtual memory techniques described above.

Interconnect innovations such as snoop filters, are essential to support scaling to higher performance memory systems. The snoop filter acts as a directory of processor cache contents and allows any memory access to be targeted directly to the processor that holds that data. More detail on this can be found in [my blog](#) on extended system coherency.

Cache Coherency Brings Heterogeneous Compute One Step Closer

HSA, with full coherency and shared virtual memory, is all about delivering new, enhanced user experiences through advances in computing architectures that bring improvements across key areas:

- performance
- power efficiency



compute potential on an SOC, where workloads can be moved seamlessly between computing devices enabling right sized computing for the given task.



Log in to like this post



By [Neil Parris](#)

Share 分享



Aa

Article
text

Copy text

Re-use is only permitted for informational and non-commercial or personal use only.



I Develop For

AI 人工智能

Automotive

Edge AI

Embedded and Microcontrollers

Laptops and Desktops

Mobile, Graphics, and Gaming

Servers and Cloud Computing

Learn

Documentation 文档

Learning Paths

On-Demand Videos

Community and Blogs



munities

Support 支持

Open a Support Case

Developer

COMMUNITY 社区

Discord 不和

Forums 论坛

Groups

Product Support Forums

Documentation 文档

Downloads 下载

Licensing 许可

Support and Training Options

CPU and Hardware

Architectures

Compare IP

Compute Subsystems

Corstone

Memory Model Tool

Performance Analysis

Processors

Reference Designs

About Arm

Leadership

Arm Offices

Careers 职业生涯

Contact Us 联系我们

Newsroom 编辑部

Trust Center

f

in

@

X

Cookie Policy 饼干的政策

Glossary

Terms of Use 使用条款

Privacy Policy 隐私政策

Accessibility 可访问性

Subscription Center

Trademarks 商标

Copyright © 1995-2025 Arm Limited (or its affiliates). All rights reserved.

